

ビルド後の処理 (PostProcessing) とスクリプト定義シンボル

PostProcessing: iOS

ビルド後の処理中に行われる内容:

- 以下のフレームワークを追加:
 - `CoreMIDI.framework`
 - `CoreAudioKit.framework`
- `Info.plist` を調整:
 - `NSBluetoothAlwaysUsageDescription` を追加

PostProcessing: Android

ビルド後の処理中に行われる内容:

- `AndroidManifest.xml` に以下の権限を追加:
 - `android.permission.BLUETOOTH`
 - `android.permission.BLUETOOTH_ADMIN`
 - `android.permission.ACCESS_FINE_LOCATION`
 - `android.permission.BLUETOOTH_SCAN`
 - `android.permission.BLUETOOTH_CONNECT`
 - `android.permission.BLUETOOTH_ADVERTISE`
- 必要な機能 (features) を追加:
 - `android.hardware.bluetooth_le`
 - `android.hardware.usb.host`

Meta Quest (Oculus Quest): USB MIDI デバイスの検出

Meta Quest デバイスで USB MIDI を使用する場合は、ビルド後の処理で USB インテントフィルタを有効にする必要があります。

`PostProcessBuild.cs` 内の、Oculus 用 USB インテントフィルタを追加する行のコメントアウトを解除してください:

```
// androidManifest.AddUsbIntentFilterForOculusDevices();
```

Android: BLE MIDI 用の CompanionDeviceManager

BLE MIDI デバイスの接続に Android の Companion Device Pairing を使用できます。

有効にする方法:

- スクリプト定義シンボルを追加: `FEATURE_ANDROID_COMPANION_DEVICE`
- Unity でのパス: `Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols`

注意点:

- Meta Quest デバイスで Bluetooth MIDI デバイスを検索・接続するためにこの機能を使用できます。
- Android のバージョンや挙動によっては、位置情報の許可リクエストが必要になる場合があります。
- Unity 6 以降（2023.1 以降）で **Application Entry Point** に **GameActivity** が含まれる場合、ビルド後処理はメイン Activity を `jp.kshoji.unity.midi.BleMidiUnityGamePlayerActivity` に設定します。**Activity** のみの場合は `jp.kshoji.unity.midi.BleMidiUnityPlayerActivity` を使用します。両方のエントリーポイントが有効な場合は、各 Unity ランチャー Activity を対応する BLE MIDI Activity に書き換えます。

Nearby Connections MIDI (Google Nearby)

依存パッケージの追加

Unity Package Manager で:

- **+** をクリック
- **Add package from git URL...** を選択
- 以下のいずれかを入力:
 - `git+https://github.com/kshoji/Nearby-Connections-for-Unity`
 - (SSH の場合) `ssh://git@github.com/kshoji/Nearby-Connections-for-Unity.git`

既にインストール済みの場合は、最新にアップデートしてください。

スクリプト定義シンボルの有効化

以下を追加:

- `ENABLE_NEARBY_CONNECTIONS`

Android プロジェクト設定

Target API level を 33 以上に設定してください:

- `Project Settings > Player > Identification > Target API Level`

使用方法の概要

アドバタイズ (公開):

- `MidiManager.Instance.StartNearbyAdvertising()`
- `MidiManager.Instance.StopNearbyAdvertising()`

ディスカバリ (検索):

- `MidiManager.Instance.StartNearbyDiscovering()`
- `MidiManager.Instance.StopNearbyDiscovering()`

接続後は、通常の MIDI データと同じ方法で送受信できます。

Maestro / MPTK 統合 (オプション)

このプラグインには、**Maestro / MidiPlayerTK (MPTK)** 用のオプションの統合レイヤーが含まれています。

有効にするには、以下のスクリプト定義シンボルを追加してください:

- `FEATURE_USE_MPTK`

Unity でのパス:

- `Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols`

このシンボルによって有効になる機能:

- MPTK をバックエンドとする **仮想 MIDI 出力** デバイス (`MidiManager` の送信を MPTK シンセにルーティング)。
- MPTK プレイヤーを **仮想 MIDI 入力** ソースとして扱うアダプター (`MidiManager` にイベントを注入)。

注意点:

- プロジェクトに MPTK アセットが存在する場合のみ、このシンボルを有効にしてください。存在しない場合、MPTK の型が見つからずコンパイルエラーになります。
- 詳細は [Maestro / MPTK 統合](#) を参照してください。

Unity 統合パッケージ（オプション）

Timeline および Visual Scripting 統合は、対応する Unity パッケージのインストールと、スクリプト定義シンボルの追加の **両方** が必要です。

Animator 統合は追加設定不要です（コア asmdef `jp.kshoji.midi` に含まれます）。

Unity でのシンボル設定パス:

- Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

Timeline 統合

1. Package Manager で **Timeline** をインストール:

- `com.unity.timeline`

2. スクリプト定義シンボルを追加:

- `FEATURE_USE_TIMELINE`

このシンボルによって有効になる機能:

- Assembly Definition `jp.kshoji.midi.timeline` / `jp.kshoji.midi.timeline.editor`
- `MidiPlaybackTrack` / `MidiRecordTrack` 等の Timeline 連携コンポーネント
- サンプルシーン `MidiTimelineIntegrationSampleScene`

注意点:

- パッケージをインストールせずにシンボルのみ有効にすると、`Unity.Timeline` 参照エラーになります。
- シンボルを無効にすると、Timeline 統合コードはコンパイル対象外になります（コアプラグインはそのままビルド可能）。

Visual Scripting 統合

1. Package Manager で **Visual Scripting** をインストール:

- `com.unity.visualscripting`

2. スクリプト定義シンボルを追加:

- `FEATURE_USE_VISUALSCRIPTING`

このシンボルによって有効になる機能:

- Assembly Definition `jp.kshoji.midi.visualscripting`
- `MidiVisualScriptingBridge` と MIDI カスタムノード群
- サンプルシーン `MidiVisualScriptingIntegrationSampleScene`

注意点:

- パッケージをインストールせずにシンボルのみ有効にすると、`Unity.VisualScripting` 参照エラーになります。
- シンボルを無効にすると、Visual Scripting 統合コードはコンパイル対象外になります。

Scriptable Audio Pipeline 統合

1. **Unity 6000.3 LTS** 以降を使用（WebGL 以外）
2. スクリプト定義シンボルを追加:
 - `FEATURE_SCRIPTABLE_AUDIO`
3. オプションパッケージ（本リポジトリの `Packages/manifest.json` に同梱済みの場合は追加不要）:
 - `com.unity.burst`
 - `com.unity.collections`

このシンボルによって有効になる機能:

- Assembly Definition `jp.kshoji.midi.scriptableaudio`
- `ScriptableAudioBootstrap` / `MidiMetronomeGenerator` / `MidiDspClockBridge` 等の基盤 API
- DSP 同期 SMF 再生: `MidiDspSequenceScheduler` / `MidiSequenceSynthGenerator` / `MidiDspSequenceBootstrap`
- DSP 同期 UMP 再生: `MidiDspUmpSequenceScheduler` / `UmpSequenceSynthGenerator` / `MidiDspUmpSequenceBootstrap`
- サンプルシーン `ScriptableAudioMetronomeSampleScene` / `ScriptableAudioSequenceSampleScene` / `ScriptableAudioUmpSequenceSampleScene`

有効化後のクイックスタート:

1. **メトロノーム**: 空の GameObject に **Scriptable Audio Bootstrap** を追加するか、`ScriptableAudioMetronomeSampleScene.unity` を開く
2. Play モード — 120 BPM のメトロノームクリックが聞こえることを確認

SMF シーケンス（DSP 同期再生）:

1. **Midi Dsp Sequence Bootstrap** を追加し `MidiSequenceAsset` を割り当てるか、`ScriptableAudioSequenceSampleScene.unity` を開く
2. Play モード — Unity DSP クロック上でノートがスケジュールされ、参照シンセで再生される

UMP シーケンス（DSP 同期再生）:

1. **Midi Dsp Ump Sequence Bootstrap** を追加し `UmpSequence` を割り当てるか、`ScriptableAudioUmpSequenceSampleScene.unity` を開く
2. Play モード — UMP クリップのノートが DSP クロック上でスケジュールされ、参照シンセで再生される
3. （任意）`.midi2` は **Assets > Create > MIDI > Import UMP Sequence Asset From File** で `UmpSequenceAsset` 化し、`ToUmpSequence()` の結果を Bootstrap に渡す
4. （任意）ハードウェア MIDI 2.0 ミラーは Bootstrap / サンプル UI の UMP Out を有効化し、事前に `MidiManager.InitializeMidi2()` を呼ぶ

注意点:

- シンボル未設定・Unity 6.2 以前・WebGL では統合 asmdef はコンパイルされません（コア `jp.kshoji.midi` は従来どおりビルド可能）。
- **SmfPlayer**（フレーム駆動・MIDI デバイス出力）と **MidiDspSequenceScheduler**（DSP サンプル精度の内蔵音声）は別コンポーネントです。用途に応じてどちらか一方を選択してください。
- **UmpSequencer**（ウォールクロック・専用スレッド）と **MidiDspUmpSequenceScheduler**（DSP 同期内蔵音声）は別コンポーネントです。クリップ編集・試験再生は前者、ゲーム内 BGM / ループ / Seek は後者を推奨します。
- UMP ハードウェア出力は `MidiManager.SendMidi2RawUmp`（フレーム粒度）。参照シンセは System メッセージを発音しません。
- エディタ診断: **Window > MIDI > Validate Scriptable Audio Setup**（メトロノーム / SMF / UMP Bootstrap を自動判別）
- 詳細: [Unity エコシステム統合 — Scriptable Audio / UMP シーケンス再生 / Scriptable Audio 統合](#)

Animator 統合

追加パッケージ・スクリプト定義シンボルは不要です。

詳細は [Unity エコシステム統合](#) を参照してください。

ネットワーク オプションキット

ネットワーク MIDI 同期は、スクリプト定義シンボルの追加が必要です。

Unity でのシンボル設定パス:

- Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

ネットワーク MIDI 同期

スクリプト定義シンボル:

- FEATURE_MIDI_NETWORK

有効になる機能:

- Assembly Definition jp.kshoji.midi.net
- MidiNetworkHub / MidiNetworkClient / MidiPlaybackSync
- サンプル MidiNetworkJamSampleScene

注意点:

- UDP ポート（既定 55000–55002）がファイアウォールで許可されている必要があります。
- シンボルを無効にすると、該当コードはコンパイル対象外になります。

オプション: Mirror / Netcode / WNet2 ブリッジ

FEATURE_MIDI_NETWORK に加え、各シンボル（とパッケージゲート）があるときだけオプションブリッジがコンパイルされます。フレームワーク本体は 同梱しません。

シンボル	説明
FEATURE_MIRROR	Mirror（MidiMirrorBridge）。あわせて MIRROR（Mirror 側）
FEATURE_NETCODE	Netcode for GameObjects（MidiNetcodeBridge）。MIDI_HAS_NETCODE は UPM versionDefines
FEATURE_WSNET2	WNet2（MidiWnet2Bridge）。MIDI_HAS_WSNET2 は WNet2.Runtime.asmdef 配置時（Editor 同期）

CI（BatchCompileBuilder）は companion として FEATURE_MIDI_NETWORK を付与します。ゲート不在時はブリッジ asm が除外され、core net はビルド可能です。

流れ: フレームワーク導入 → シンボル → コンポーネント配線 → サンプル。詳細は [ジャンルキット](#)、[統合](#)、[Networking 統合](#)。

UDP ベースの `MidiNetworkHub` / `MidiNetworkClient` は上記なしでも利用できます。

横断基盤 オプションキット

Input System ブリッジはスクリプト定義シンボルの追加が必要です。

MidiClockSync / 和音・スケール判定はシンボル不要です（コア `jp.kshoji.midi`）。

Unity でのシンボル設定パス:

- Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

Input System ブリッジ

1. Package Manager で `com.unity.inputsystem` を追加（`Packages/manifest.json` に同梱済みの場合は不要）
2. スクリプト定義シンボルを追加:
 - `FEATURE_INPUT_SYSTEM`

有効になる機能:

- Assembly Definition `jp.kshoji.midi.inputsystem`
- `MidiInputSystemBridge` / `InputSystemToMidiBridge` / `MidiSyntheticDevice`
- サンプル `InputSystemBridgeSampleScene`

注意点:

- シンボルを無効にすると、Input System 統合コードはコンパイル対象外になります（コアプラグインはそのままビルド可能）。

MidiClockSync / 和音・スケール判定

追加シンボルは不要です。コア `asmdef`（`jp.kshoji.midi`）に含まれます。

詳細は [ジャンルキット — 横断基盤](#) を参照してください。

Chunity (Chuck) 統合（オプション）

本プラグインは **Chunity** ランタイムを同梱しません。利用者が別途インストールし、シンボルを有効にしたときだけ連携コードがコンパイルされます。

1. [Chunity](#) をプロジェクトヘインストール
2. `Assets/MIDI/Scripts/Integrations/Chunity/Optional/Chunity.Runtime.asmdef.example` を Chunity の Scripts ルートへ `Chunity.Runtime.asmdef` としてコピー（例: `Assets/Chunity/Scripts/Chunity.Runtime.asmdef`）。詳細は [Chunity Optional パッチ手順](#)
3. スクリプト定義シンボルを追加:
 - `FEATURE_CHUNITY`

有効になる機能:

- Assembly Definition `jp.kshoji.midi.chunity`
- `MidiChuckBridge` / `MidiChuckPatchHost` / `MidiChuckMapping` / Phase 2-3 コンポーネント
- サンプル `ChunityBridgeSampleScene` / `ChunityWorkflowsSampleScene` / `ChunityPresetsSampleScene`
- （任意）`FEATURE_USE_TIMELINE` → `jp.kshoji.midi.chunity.timeline`
- （任意）`FEATURE_USE_VISUALSCRIPTING` → `jp.kshoji.midi.chunity.visualscripting`

Scriptable Audio Generator（追加オプション）

Chuck の音声出力を `OnAudioFilterRead` から Unity 6.3+ の Scriptable Generator に差し替えます（制御パスは変更なし）。

1. [Chunity Optional パッチ手順](#) の手順で、Chunity 本体に `useBuiltInAudioFilter` パッチを適用
2. Package Manager で `com.unity.collections` を入れます（asmdef が `Unity.Collections` を参照。 `NativeArray` / `FixedString128Bytes` に必要）
3. スクリプト定義シンボルを追加:
 - `FEATURE_CHUNITY_SCRIPTABLE_AUDIO` （`FEATURE_CHUNITY` も必要）

有効になる機能:

- Assembly Definition `jp.kshoji.midi.chunity.scriptableaudio`
- `ChuckMainGeneratorDriver` / `ChuckSubGeneratorDriver`
- サンプル `ChunityGeneratorWorkflowSampleScene`

注意: WebGL および未対応環境では従来の `FilterRead` / `WebChuck` にフォールバックします。メトロノーム等の本体 Scriptable Audio 統合は別シンボル `FEATURE_SCRIPTABLE_AUDIO` です。

注意点:

- シンボルのみ有効で Chunity 未導入（または `Chunity.Runtime` `asmdef` 未配置）の場合、型解決エラーになります（MPTK と同様）。
- シンボルを無効にすると、統合コードはコンパイル対象外になります（コアプラグインはそのままビルド可能）。
- ユーザー向け説明: [Unity エコシステム統合 — Chunity](#)
- セットアップ詳細: [Chunity 統合](#)